

BIT4138: ADVANCED CRYPTOGRAPHY

Practical Logbook — Week 8

Student Name	Eddy Kering
Admission Number	BSCCS/2024/61947
Course / Class	Computer Science
Unit Code	BIT4138
Unit Name	Advanced Cryptography
Lecturer Name	Michael Nyoro
Semester / Academic Year	2026

Project Title	BIT4138 Advanced Cryptography — PKI, Diffie-Hellman and Asymmetric Cryptography
Selected Technologies	Python 3, VS Code, Cryptography Library, PyCryptodome, OpenSSL, Git, GitHub
GitHub / Portfolio Link	https://github.com/keringyegon-byte/BIT4138-Cryptography

TABLE OF CONTENTS

Cover Page.....	1
Week 8: PKI, Asymmetric Ciphers and Diffie-Hellman.....	2
8.1 Overview.....	2
8.2 Key Concepts Applied.....	2
8.3 PKI Components.....	2
8.4 Symmetric vs Asymmetric Comparison.....	3
8.5 Required Evidence.....	4
Fig 1: Diffie-Hellman Implementation Code.....	4
Fig 2: Diffie-Hellman Key Exchange Output.....	5
Fig 3: PKI Certificate Inspection (OpenSSL).....	5
Fig 4: RSA Key Generation and Certificate Signing.....	6
Fig 5: Full Secure Communication Demo.....	6
Student Reflection.....	7
Weekly Summary.....	7

Week 8: PKI, Asymmetric Ciphers and Diffie-Hellman Key Exchange

Public Key Infrastructure, Digital Certificates, Intractability, NP-Completeness and Diffie-Hellman

Overview

This week covered two major topics. The first was Public Key Infrastructure (PKI) — the framework that manages digital certificates and enables trust in public-key cryptography, including Certificate Authorities (CAs), the X.509 certificate standard, and the certificate lifecycle. The second topic was asymmetric cryptography fundamentals including computational complexity, intractability, NP-complete problems, and the Diffie-Hellman Key Exchange protocol which allows two parties to securely establish a shared secret over a public channel.

Key Concepts Applied

- PKI: a framework of policies and technologies for managing digital certificates and public-key encryption
- Certificate Authority (CA): a trusted organization that issues and signs digital certificates (e.g. DigiCert, GlobalSign)
- X.509: the standard format for digital certificates containing owner identity, public key, validity period, and CA signature
- Diffie-Hellman: a key exchange protocol allowing two parties to derive the same shared secret without prior communication
- Discrete Logarithm Problem: given $g^a \bmod p$, finding a is computationally infeasible — this secures Diffie-Hellman
- NP-Complete problems: computationally hard problems whose solutions are easy to verify but extremely hard to find

PKI Components

Component	Role	Example
Certificate Authority (CA)	Issues and signs digital certificates	DigiCert, GlobalSign, Let's Encrypt
Registration Authority (RA)	Verifies user identity before CA issues certificate	University IT department
Digital Certificate	Binds a public key to an identity	HTTPS website certificate
Certificate Repository	Stores and distributes certificates	LDAP directory, CA website
CRL / OCSP	Lists revoked or checks real-time certificate status	Browser certificate check

Symmetric vs Asymmetric — Comparison

Feature	Symmetric (AES)	Asymmetric (RSA / DH)
Keys Used	One shared key	Public + Private key pair

Speed	Fast	Slow
Key Distribution	Difficult	Easy — public key shared openly
Use Case	Bulk data encryption	Key exchange, digital signatures
Examples	AES, DES	RSA, Diffie-Hellman, ECC

Diffie-Hellman Key Exchange Steps

Public parameters agreed by both parties: prime $p = 23$, generator $g = 5$

- Step 1: Alice chooses private key $a = 6$, computes $A = 5^6 \bmod 23 = 8$
- Step 2: Bob chooses private key $b = 15$, computes $B = 5^{15} \bmod 23 = 19$
- Step 3: Alice and Bob exchange public keys ($A = 8$, $B = 19$) over public channel
- Step 4: Alice computes $S = 19^6 \bmod 23 = 2$
- Step 5: Bob computes $S = 8^{15} \bmod 23 = 2$
- Both arrive at the same shared secret $S = 2$ without it ever being transmitted

Required Evidence

Fig 1: Diffie-Hellman Implementation Code

```

16 def dh_exchange(p, g, alice_private, bob_private):
17     """Perform Diffie-Hellman key exchange and display all steps"""
18     print(f"\n Public Parameters:")
19     print(f" Prime (p)      : {p}")
20     print(f" Generator (g)      : {g}")
21     print(f"\n Private Keys (kept secret):")
22     print(f" Alice private (a) : {alice_private}")
23     print(f" Bob private (b)   : {bob_private}")
24
25     # Generate public keys
26     alice_public = pow(g, alice_private, p)
27     bob_public = pow(g, bob_private, p)
28
29     print(f"\n Public Keys (exchanged openly):")
30     print(f" Alice public A = g^a mod p = {g}^{alice_private} mod {p} = {alice_public}")
31     print(f" Bob public B = g^b mod p = {g}^{bob_private} mod {p} = {bob_public}")
32
33     # Compute shared secrets
34     alice_secret = pow(bob_public, alice_private, p)
35     bob_secret = pow(alice_public, bob_private, p)
36
37     print(f"\n Shared Secret Computation:")
38     print(f" Alice: S = B^a mod p = {bob_public}^{alice_private} mod {p} = {alice_secret}")
39     print(f" Bob : S = A^b mod p = {alice_public}^{bob_private} mod {p} = {bob_secret}")
40
41     print(f"\n Shared Secret Match : {'YES - Key exchange successful!' if alice_secret == bob_secret}")
42     print(f" Shared Secret Value : {alice_secret}")
43     return alice_secret

```

Fig 1: Python source code showing the Diffie-Hellman key exchange implementation with public parameters, private key selection, public key generation, and shared secret computation

Fig 2: Diffie-Hellman Key Exchange Output

```
Select option: 1

-----
Diffie-Hellman Key Exchange
-----

Enter prime p (default 23):
Enter generator g (default 5):
Alice private key (default 6):
Bob private key (default 15):

Public Parameters:
Prime (p)      : 23
Generator (g)   : 5

Private Keys (kept secret):
Alice private (a) : 6
Bob private (b)   : 15

Public Keys (exchanged openly):
Alice public A = g^a mod p = 5^6 mod 23 = 8
Bob public B = g^b mod p = 5^15 mod 23 = 19

Shared Secret Computation:
Alice: S = B^a mod p = 19^6 mod 23 = 2
Bob : S = A^b mod p = 8^15 mod 23 = 2

Shared Secret Match : YES - Key exchange successful!
Shared Secret Value : 2

[1] Diffie-Hellman Key Exchange (textbook example)
[2] Diffie-Hellman with Large Prime (real-world)
```

Fig 2: Terminal output showing Alice and Bob's public keys, the shared secret computed by both parties, and confirmation that both secrets match

Fig 3: PKI Certificate Inspection (OpenSSL)

```
=====
BIT4138 Advanced Cryptography - Week 8
PKI, Asymmetric Ciphers and Diffie-Hellman
=====

[1] Diffie-Hellman Key Exchange (textbook example)
[2] Diffie-Hellman with Large Prime (real-world)
[3] PKI - Generate and Inspect Certificate (OpenSSL)
[4] Full Secure Communication Demo (DH + AES)
[5] NP-Completeness Demo (Discrete Log difficulty)
[6] Exit

Select option: 3

-----
PKI - Generate RSA Key + Self-Signed Certificate
-----

Generating RSA key and self-signed certificate...
Serial Number:
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=KE, ST=Nairobi, L=Nairobi, O=BIT4138, CN=eddy.kering
Not Before: Jun 29 00:02:47 2026 GMT
Not After : Jun 29 00:02:47 2027 GMT
Subject: C=KE, ST=Nairobi, L=Nairobi, O=BIT4138, CN=eddy.kering
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
X509v3 Subject Key Identifier:
Signature Algorithm: sha256WithRSAEncryption

Certificate files saved: demo key.pem, demo cert.pem
```

Fig 3: Terminal output of OpenSSL inspecting a certificate, showing issuer, subject, validity dates, public key algorithm, and signature details

Fig 4: RSA Key Generation and CSR

```
PS C:\BIT4138_Python_Scripts> type demo_cert.pem
-----BEGIN CERTIFICATE-----
MIIDYTCCAkmgAwIBAgIUfGWR0tJtTaEQDbIEQ9GHbLVtN9cwDQYJKoZIhvcNAQEL
BQAwWTElMAkGA1UEBhMCS0UxEDAOBgNVBAGMB05haXJvYmKxEDAOBgNVBAMCMB05h
aXJvYmKxEDAOBgNVBAoMB0JJVDQxMzgxZDASBgNVBAMMC2VkZHkua2VyaW5nMB4X
DTI2MDYyOTAwMDI0N1oXDTI3MDYyOTAwMDI0N1owWTElMAkGA1UEBhMCS0UxEDAO
BgNVBAGMB05haXJvYmKxEDAOBgNVBAMCMB05haXJvYmKxEDAOBgNVBAoMB0JJVDQx
MzgxZDASBgNVBAMMC2VkZHkua2VyaW5nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A
MIIBCGKCAQEAw702GfxNgxw9Hzj4yWo0/6TwnJJ1uLLIW/kFfuOEvvxN75QjbAMF
w1oUd8HJEW0cMchrJ3bT9wXemOEm9GKjXNR5GqY4rg1aJdt9j+Rr5a1g1Thgprdd
cw2k6TdcQZ7q3ld/iavX07PFhu30NxH+ftKONAWdKHTWfpPrfTvxx+xd0T8DFYOLu
PVqhkoBvz4eE81aElpCvx3cxdcIUbIOH8PVGaKuxb1UnsFhmYWVc6Z24nJcZI8gP
0H3YhrYyW94mYcrp43fBLjuLQwzoA20Z71zxr8Cr7OE0tDaGkGK40hsY/P9C3VUW
nM2DW5XWoXy9U+YpBrbWnGAU+0mLmOeczQIDAQABoyEwHzAdBgNVHQ4EFgQUoDXZ
Gv9wdPPILWufGo67sUTjbOYwDQYJKoZIhvcNAQELBQADggEBAA/rSbkgkSWCDOjY
xsf2t4/xLXDA1Y2+md1e9sZV648nRxxrmMNjmf+/xhZIrTmsk4wOXkqlQ09NxJWpZ
6Mf0JRVAIyvVtvHJWG8jD4T1/Qk0TbKEXA0z0tesMYAWLp2YpieaI6aHxPGRXbYG
W1xERgk8C7Xotl1QkYpV5HxXuXGVDwmPfg0wzoxe716IFzrlUpB1ymljb06aZlgn
F1fDTjv1r2VjIueKq4a/nwP7GWJePcGRtMeeb3pIcz/3g0dklWOKLqwu+j+cfNKRz
Wx5hv14SqI/w4zjEpFPCIXBKVGfNSBzNQEFUx3W6uci7LabFXTSDgI6aXjV8bti4
X1JkSbc=
-----END CERTIFICATE-----
PS C:\BIT4138_Python_Scripts>
PS C:\BIT4138_Python_Scripts>
```

Fig 4: Terminal output showing RSA key pair generation and a Certificate Signing Request (CSR) being created with OpenSSL, demonstrating the PKI certificate lifecycle

Fig 5: Full Secure Communication Demo

```
[1] Diffie-Hellman Key Exchange (textbook example)
[2] Diffie-Hellman with Large Prime (real-world)
[3] PKI - Generate and Inspect Certificate (OpenSSL)
[4] Full Secure Communication Demo (DH + AES)
[5] NP-Completeness Demo (Discrete Log difficulty)
[6] Exit

Select option: 4

-----
Full Secure Communication Demo
-----

Enter message to send securely: Hello Bob, this message is encrypted!

Message to send securely: 'Hello Bob, this message is encrypted!'

Step 1: Diffie-Hellman key exchange...
Shared secret = 2

Step 2: Alice encrypts message with AES using shared secret...
Ciphertext (hex): eb874ab0b319d97d853f39767c68a152af4cecd8bc8bc5b1d...

Step 3: Bob decrypts using the same shared secret...
Decrypted message: 'Hello Bob, this message is encrypted!'

Secure communication: SUCCESS
```

Fig 5: Terminal output demonstrating a complete secure communication flow: Diffie-Hellman key exchange followed by AES encryption of a message using the derived shared secret

Student Reflection

PKI and Diffie-Hellman together solve the key distribution problem in public-key cryptography. PKI provides trust through Certificate Authorities, ensuring a public key truly belongs to its claimed owner. Diffie-Hellman enables two parties to derive an identical secret over an insecure channel. Both rely on computational hardness — discrete logarithms and integer factorization — to guarantee security.

Weekly Summary

This week I implemented Diffie-Hellman key exchange in Python, verified correct shared secret derivation by both parties, and inspected a digital certificate using OpenSSL. The PKI certificate lifecycle was studied covering generation, signing, distribution, and revocation. Diffie-Hellman security depends on the discrete logarithm problem, making it computationally infeasible to recover private keys from observed public values.